Recod.ai/LUC - Scientific Image Forgery Detection

Max Bakke Seymour[1], Einar Martin Søvik Bernsen[1], and Emre Balci[1]

[1]University of Bergen (UiB)

Abstract

TODO - "Short summary of the paper, including results. Often written last".

1 Introduction

1.1 Copy-Move Forgery Detection

Copy-Move Forgery is a class of image forgery where regions of an image are duplicated and moved to other areas of the image. Commonly, techniques such as rotation, flipping and scaling of the copied region are applied, as well as post-processing such as edge smoothing, brightness adjustment and adding extra noise. These techniques can lead to forgeries which are hard to detect both through manual review and automated detectors. This method of forgery can be used within scientific images in order to fabricate results, where the complexity of the figures creates additional difficulties. [1]

1.2 Related works

Copy-move image forgery detection has been studied through the lens of many different techniques. Early approaches focused on hand-crafted descriptors extracted either densely over overlapping blocks or sparsely at keypoints, followed by nearest-neighbour matching and geometric consistency checks [2, 3]. These methods established the basic copy-move detection pipeline and showed that explicit correspondence search can be effective, but they are often sensitive to strong post-processing, complex geometric transformations, and repeated structures in the scene [3].

In more recent work, the focus has shifted towards deep learning for copy-move detection. Some methods retain the classical matching intuition while replacing hand-crafted features with learned representations and stronger dense matching modules [4]. Others formulate the task as end-to-end forgery localization, using CNN-based architectures to jointly detect duplicated content and localize manipulated regions [5]. Attention-based models have further improved localization by explicitly modelling patch relationships and co-occurrence cues [6], while more recent transformer-based approaches have explored global context modelling for copy-move detection [7]. In parallel, source-target disambiguation has also emerged as an important subproblem, since detecting near-duplicate regions alone is not always sufficient to identify the truly manipulated area [8].

One of the other architectures we tried for this task even combines both perspectives by using CNN crafted features in combination with features computed through Zernike kernels. [9]

1.3 Objectives

We aim to create a full prediction pipeline for CMFD which could perform well on the Recod.AI Scientific Image Forgery Detection dataset. Ideally, our model should be robust towards post-processing techniques. The main goal is to perform well on the competition metric, which is a modified version of F1 which computes pixelwise F1 on pairs of (*predicted mask, ground truth mask*) using the following formula [10]:

$$F1 = \frac{2TP}{2TP + FP + FN}$$

This calculation is applied to every connected component of forged pixels in the image (belonging to either the source or target). Predictions given by the model are matched up with the best corresponding ground-truth areas using the Hungarian algorithm. For any additional predicted areas predicted by the model above the actual amount of ground-truth components, a penalty is computed:

$$P = \frac{n_{gt}}{\max(n_{pred}, n_{gt})}$$

Finally, the per-image score is given as:

$$oF1 = \text{mean(best matched pairwise F1 scores)} \times P$$

The competition metric is challenging for a few reasons. Firstly, it requires high pixel-level precision for the output masks. In addition, a score of 0 for a given image is given if any pixels are predicted as forged on a ground-truth authentic image. Finally, it is highly punishing if any additional components are predicted, even if they should be small, as this affects the penalty P.

1.4 Contributions

This paper presents a complete main pipeline for CMFD and several alternate methods, discussing the strengths and weaknesses of the different approaches. Through a discussion and presentation of these methods, we aim to identify what directions could be promising for future work and further nuance the task of CMFD.

2 Methods

Our initial implementation was inspired by a notebook by Kaggle user Gaurav Parkhedkar [11], who successfully employed a range of different techniques to perform a score of 0.332 on the competition metric on the test set. These techniques include feature extraction through the DINOv2 model [12], sliding window inference and prediction mask post-processing. For our own implementation we build on these techniques to try other possibilities.

2.1 Dataset

We utilize the dataset supplied in the Kaggle competition [1]. Unfortunately, we were unable to test our implementation against the Kaggle test-set, as we were too late to enter the competition. As we did not have access to the test data, we instead chose to use the available training data in a train (80%), evaluate (10%), test (10%) split. The training data consists of a base 2751 forged and 2377 authentic images as well as an additional 48 forged images. As the images are too large to process directly with the model, they are each resized to size $3 \times 448 \times 448$.

2.2 Feature Extraction

To perform feature extraction on the images, we employ Meta AI's powerful DINOv2 model using the pretrained *ViT-B/14* segmentation head [12]. This model first extracts strong general-purpose features using the DINOv2 ViT backbone, then uses these features to create pixel-level predictions using the segmentation head. This model is kept frozen and applied on 14×14 size patches of the image at a time to generate features with an embedding dimension of 768. When predicting,

2.3 Decoder

In order to create the predicted pixel mask, we employ a small decoder model on the generated DINOv2 feature map. The decoder we utilize contains three blocks and an output layer, before outputting the generated mask interpolated to output size using bilinear interpolation.

The first two blocks of the decoder apply convolutions to bring the dimensionality from $768 \to 384 \to 192$ while also applying $ReLU$ and *Dropout* with $p = 0.1$ operations in each block. Then in the third block a convolution from $192 \to 96$ is applied before a final $ReLU$ and the output layer $96 \to 1$ which gives the predicted mask.

2.4 Mask post-processing

To post-process masks, we applied the following operations to an input map of forged pixel probabilities:

1. Gaussian blur: A smoothing operation implemented through `scipy.ndimage.gaussian_filter(mask, sigma=0.5)`
2. Confidence threshold filtering: Creates an initial binary mask by filtering probabilities on the threshold 0.5.
3. Restore very confident pixels: Restores pixels predicted with a confidence ≥ 0.9, as they may have been lost through smoothing, and combines them with the mask generated through confidence threshold filtering.
4. Closing operation: Applies dilation followed by erosion in order to create more coherent objects, implemented through `scipy.ndimage.binary_closing(mask, structure=np.ones((5, 5)))`
5. Opening operation: Applies erosion followed by dilation in order to remove small isolated blobs, implemented through `scipy.ndimage.binary_opening(mask, structure=np.ones((3, 3)))`
6. Finally, filling any remaining holes in binary objects using `scipy.ndimage.binary_fill_holes(mask)`

2.5 Sliding window inference

In order to learn global relationships between pixels in an image, it is necessary to run inference on images as a whole. This is performed on images of the same resized size as the training image input size. The generated pixel masks are upscaled through bilinear interpolation, similarly to the training image samples.

The issue with this kind of inference is that although it is able to learn the global relationships between objects in the image, it is challenging to achieve perfect pixel-level precision on images when the resolution is decreased significantly. In order to give our model more pixel-level precision and allow for inference on higher-resolution images directly, we employ sliding window inference. Sliding

window inference is performed by performing individual predictions on overlapping patches of the image, which are weighted by a gaussian kernel in order to create a final output pixel map. TODO: More detailed: This is combined with the global prediction performed on the whole downscaled image at once.

3 Results

TODO: Show results of experiments. NB: Without interpretation at this stage!

Align with method section.

4 Discussion

TODO: Interpret results, relate to objectives (did we achieve them?), compare against others

One issue with this task is loss function misalignment. Although cross-entropy loss correctly penalizes incorrect pixel-level predictions, it is not fully aligned with the strict conditions that the oF1 metric expects. For instance, if a small area of forged pixels is predicted on an authentic image, the impact on the loss is not large, but this makes the oF1 metric 0. This meant that we in the process of training different models observed that from epoch to epoch, even if the overall validation loss decreased, it could be the case that performance in regard to the oF1 metric became worse.

Although beyond the scope of this project, an interesting avenue for future research could be to create a differentiable loss more directly linked to the OF1 objective. As surrogates, we did employ additionally loss penalties for any forged pixels predicted on authentic images, but a more nuanced alternative would be interesting to experiment with.

Because of this mismatch, model performance is very sensitive to the operations performed in mask post-processing. We see experimenting with other post-processing techniques as a promising direction for future work.

4.1 Deep Cross-Scale PatchMatch

The Deep Cross-Scale PatchMatch architecture was proposed by Li et al. in 2024. [9] It is an end-to-end differentiable deep-learning based architecture where one branch employs the patch match algorithm presented by Barnes et al. in 2009 [13] on different scales of each image in order to attain pixel similarities and offsets from each pixel to it's most similar pixel. The second branch runs a direct feature extraction → prediction process. Finally, the predictions of the two branches are combined through an argmax operation. The architecture supports three-channel CMFD, but is adapted to the single-channel CMFD by only using the top branch. Our implementation has some differences from the version by Li et al. This is partly due to practical concerns, and partly due to ambiguity or lack of detail in the paper regarding specific implementation details.

Most changes were performed in the first branch, where one change performed due to practical concerns was to use a frozen pretrained CNN feature extractor. This meant that it was not necessary to let gradients flow through the DLF and Deep Cross-Scale PatchMatch parts of the networks, saving memory and computation time. We tested using both the pretrained VGG16 and ResNet18 weights from `torchvision.models` [14, 15], achieving best performance with ResNet18. In addition, although the CNN is run multi-scale in the paper, we found that the difference in performance when using pretrained models was marginal, meaning it was more useful to save memory and time by running this model single-scale while still keeping Zernike feature extraction multi-scale. Inspired by its strong performance in our baseline, we performed feature extraction using DINOv2 in the second branch of the architecture.

TODO: Another practical change was in adjusting the loss function, where we included more loss terms than were used in the paper.

The main weaknesses of this architecture are in runtime and memory consumption. The runtime is particularly dependent on the number of PatchMatch iterations which are performed. This module recurrently executes the propagation and evaluation layers "several times" [9, p. 7]. It is not clear how many iterations the authors employed, but in order to attain decent offsets, we found that a suitable amount was 24 iterations. Using this amount of iterations, PatchMatch on a single image takes 4s on an NVIDIA RTX 4070 TI Super GPU, while other components of the network take less than $0.1s$ to run.

The final performance we were able to achieve on the hold-out test-set using this method was TODO. We assess that several factors contributed to the worse performance of this architecture. Firstly, the forged images in the dataset often contain the same

object copied multiple times (TODO: EXAMPLE). As each pixel only keeps it's most similar offset in the PatchMatch branch, this means that any object which is copied more than once risks unstable performance within this branch, as pixels within the same object may be matched to pixels within different copied instances, causing high DLF errors and a poor signal to the decoder. An improved implementation for future work could for instance experiment keeping the top-k matches per pixel instead of only one. In contrast, this is not an issue for purely feature-extraction based approaches such as our main pipeline.

PatchMatch's reliance on finding groups of pixels with high feature similarity may have some other fundamental issues in regards to this particular dataset. In the paper, the architecture trains on a training set generated from MS COCO [16] by applying a random copy-move, rotation and scaling operation to a random area. It is then evaluated on a synthetic dataset generated in the same way as well as CASIA CMFD, CoMoFoD and CMH [17–19]. The common denominator for these datasets is that they feature photo-realistic images with varied objects and backgrounds. This means that there is natural variation in the qualities of pixels (for instance, they do not feature a consistent single-colour background) in contrast to many of the scientific images, which are dark, or figures containing a consistent background in which many false positives are generated from pixel-wise similarities.

4.2 BusterNet

TODO: BusterNet Comments

4.3 Comments on Kaggle submissions

At the time of writing, the maximum oF1 score achieved on hold-out test data was 0.458, achieved by Kaggle user *orshanec*. [1] We again emphasize that our architecture cannot be directly compared with this metric due to us being unable to access the Kaggle hold-out test set.

References

[1] Kaggle. *Recod.ai/LUC - Scientific Image Forgery Detection*. https://www.kaggle.com/competitions/recodai-luc-scientific-image-forgery-detection/overview. [Online; last accessed April-2026]. 2025.

[2] J. Fridrich, D. Soukal, and J. Lukáš. "Detection of Copy-Move Forgery in Digital Images". In: (2003). URL: https://ws2.binghamton.edu/fridrich/Research/copymove.pdf.

[3] V. Christlein, C. Riess, J. Jordan, C. Riess, and E. Angelopoulou. "An Evaluation of Popular Copy-Move Forgery Detection Approaches". In: *IEEE Transactions on Information Forensics and Security* 7.6 (2012), pp. 1841–1854. DOI: 10.1109/TIFS.2012.2218597. URL: https://arxiv.org/abs/1208.3665.

[4] Y. Liu, C. Xia, X. Zhu, and S. Xu. "Two-Stage Copy-Move Forgery Detection With Self Deep Matching and Proposal SuperGlue". In: *IEEE Transactions on Image Processing* 31 (2022), pp. 541–555. DOI: 10.1109/TIP.2021.3132828. URL: https://doi.org/10.1109/TIP.2021.3132828.

[5] Y. Wu, W. Abd-Almageed, and P. Natarajan. "BusterNet: Detecting Copy-Move Image Forgery with Source/Target Localization". In: *Computer Vision – ECCV 2018* (2018), pp. 170–186. URL: https://openaccess.thecvf.com/content_ECCV_2018/papers/Rex_Yue_Wu_BusterNet_Detecting_Copy-Move_ECCV_2018_paper.pdf.

[6] A. Islam, C. Long, A. Basharat, and A. Hoogs. "DOA-GAN: Dual-Order Attentive Generative Adversarial Network for Image Copy-Move Forgery Detection and Localization". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (2020), pp. 4676–4685. DOI: 10.1109/CVPR42600.2020.00473. URL: https://doi.org/10.1109/CVPR42600.2020.00473.

[7] Y. Liu, C. Xia, S. Xiao, Q. Guan, W. Dong, Y. Zhang, and N. Yu. "CMFDFormer: Transformer-based Copy-Move Forgery Detection with Continual Learning". In: *CoRR* abs/2311.13263 (2023). DOI: 10.48550/ARXIV.2311.13263. URL: https://arxiv.org/abs/2311.13263.

[8] M. Barni, Q.-T. Phan, and B. Tondi. "Copy Move Source-Target Disambiguation Through Multi-Branch CNNs". In: *IEEE Transactions on Information Forensics and Security* 16 (2021), pp. 1825–1840. DOI: 10.1109/TIFS.2020.3045903. URL: https://doi.org/10.1109/TIFS.2020.3045903.

[9] Y. Li, Y. He, C. Chen, L. Dong, B. Li, and J. Zhou. "Image Copy-Move Forgery Detection via Deep PatchMatch and Pairwise Ranking Learning". In: (2024). Also available at: https://arxiv.org/pdf/2404.17310. URL:

https://ieeexplore.ieee.org/document/10736426.

[10] Kaggle. *RecodAI F1*. https://www.kaggle.com/code/metric/recodai-f1/. [Online; accessed January-2026]. 2025.

[11] G. Parkhedkar. *DINOv2 Base [0.332] — High-Res 4500px — Robust Inf*. https://www.kaggle.com/code/gauravparkhedkar/dinov2-base-0-332-high-res-4500px-robust-inf. [Online; accessed January-2026]. 2026.

[12] M. Oquab, T. Darcet, T. Moutakanni, H. V. Vo, M. Szafraniec, V. Khalidov, P. Fernandez, D. Haziza, F. Massa, A. El-Nouby, et al. "DINOv2: Learning Robust Visual Features without Supervision". In: *arXiv preprint arXiv:2304.07193* (2023). implementation: https://github.com/facebookresearch/dinov2. URL: https://arxiv.org/abs/2304.07193.

[13] C. Barnes, E. Shechtman, A. Finkelstein, and D. B. Goldman. "PatchMatch: A Randomized Correspondence Algorithm for Structural Image Editing". In: (2009). URL: https://dl.acm.org/doi/10.1145/1531326.1531330.

[14] K. Simonyan and A. Zisserman. "Very Deep Convolutional Networks for Large-Scale Image Recognition". In: *arXiv preprint arXiv:1409.1556* (2014). URL: https://arxiv.org/abs/1409.1556.

[15] K. He, X. Zhang, S. Ren, and J. Sun. "Deep Residual Learning for Image Recognition". In: *arXiv preprint arXiv:1512.03385* (2016). URL: https://arxiv.org/abs/1512.03385.

[16] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick. "Microsoft COCO: Common Objects in Context". In: *European Conference on Computer Vision (ECCV)*. 2014, pp. 740–755.

[17] J. Dong, W. Wang, and T. Tan. "CASIA Image Tampering Detection Evaluation Database". In: *IEEE China Summit and International Conference on Signal and Information Processing*. 2013, pp. 422–426.

[18] D. Tralic, I. Zupancic, S. Grgic, and M. Grgic. "CoMoFoD — New Database for Copy-Move Forgery Detection". In: *Proceedings of the International Symposium ELMAR*. 2013, pp. 49–54.

[19] E. Silva, T. Carvalho, A. Ferreira, and A. Rocha. "Going Deeper into Copy-Move Forgery Detection: Exploring Image Telltales via Multi-Scale Analysis and Voting Processes". In: *Journal of Visual Communication and Image Representation* 29 (2015), pp. 16–32.

A Appendix Example